Day 2: Application Containerization
- Dockerfile deep dive (multi-stage builds, caching)
- Containerizing monolithic apps
- Application environment configurations
- Debugging container builds
- Hands-On: Containerize a legacy app (Node/Java/Python), optimize Dockerfile

1. Dockerfile Deep Dive:

Definition: A Dockerfile is a script containing instructions to build a container image.

```
FROM node:18
WORKDIR /app
COPY ...
RUN npm install
CMD ["node","app.js"]
```

Instruction	Purpose
FROM	Base image
WORKDIR	Working directory
COPY	Copy files from local + container directory
RUN	Install dependencies
CMD	Start application

Instruction	Purpose
FROM	Defines base image (starting point)
WORKDIR	Sets working directory inside container
COPY	Copies files from local → container
RUN	Executes commands during build
CMD	Defines default command when container starts

*** Execution Flow ***

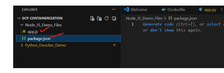
(1) Base Image → (2) Create Directory → (3)Copy Code → (4)Install → (5)Run App

*** Real-World Example ***

A Node.js application:

package.json = dependencies

app.js = main code



** Dockerfile ensures:
Same environment everywhere - Docker will be initialized and it starts building here

** Multi-Stage Builds **

❗ Problem

-> Traditional Docker images:

-- Include build tools (Node, compilers, etc.)

-- Become large and inefficient

Solution: Multi-Stage Build

. . . . . .

FROM node:18 AS builder

WORKDIR /app
COPY ...
RUN npm install && npm run build

FROM nginx

COPY --from=builder /app/build /usr/share/nginx/html

COPY --> Standard Docker command to copy files.
--> from=builder --> This tells Docker: "Copy files from the builder stage, not from local system"
/app/build --> Source path inside builder container (this is created by : npm run build)
/usr/share/nginx/html --> Destination inside Nginx container,
-- Default folder where Nginx serves static files from

output

How It Works:

Stage 1 (builder):
- Installs dependencies
- Builds application

Stage 2 (production):
- Only copies final output
- No unnecessary tools

** Outcome:
-- Smaller image size
-- Faster deployment
-- Better security
-- Reduced attack surface - (reduces the attackers / security risk is less)
-- Lower risk at Production with less tools / packages

** Real-World Scenario **

Frontend React app:

Build stage → creates /build folder
Production → only serves static files via Nginx

Step1: --> nano app.py

CTRL + X + Y + ENTER

Step2: --> nano requirements.txt write : flask

Step3: Create a Dockerfile

Step 4: Build Docker Image

docker build -t myapp .

Step 5: Run Container

*** Docker Caching ***

Docker builds images in layers

Each line in a Dockerfile = one layer

. . . . . .

FROM node:18
COPY package.json ...
RUN npm install
COPY ...

These become layers like:

Layer 1 = Base image
Layer 2 = package.json
Layer 3 = npm install
Layer 4 = app code

How Caching Works?:

Docker checks each step:

If no change → reuse cached layer - Done

If change detected → rebuild from that step - Rebuild

- Dependencies get cached
- Only code layer rebuilds

** Industry Scenario **

Legacy Banking App
Runs on physical server
Deployment = slow

-> We Do the containerization
* After containerization:
- Faster deployments
- Easy scaling in cloud
- Portable across environments

Demos: Node.js Monolithic App

```
Successfully built 890d413ba264
Successfully tagged node-app:latest
ajejkumar38/7days-instance-20240314-093931:~/monolithic-ap-node$sd sudo docker run -d -p 3000:3000 node-app
ae97c2f24a2ba8f7f4b9e7c2f17f7c2a4a1b8cfc7522f6929d71f2b4f1aa8bc41
ajejkumar38/7days-instance-20240314-093931:~/monolithic-ap-node$sd node docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                    NAMES
ae97c2f24a2b   node-app  "docker-entrypoint.s.."  9 seconds ago  Up 8 seconds  0.0.0.0:3000->3000/tcp,  :::3000->3000/tcp   zad_mcclintock
d5f413b0d9ab   myapp     "python app.py"          3 hours ago    Up 3 hours    0.0.0.0:5000->5000/tcp,  :::5000->5000/tcp   modest_pare
ajejkumar38/7days-instance-20240314-093931:~/monolithic-ap-node$sd curl http://localhost:3000
<!DOCTYPE html>

ajejkumar38/7days-instance-20240314-093931:~/monolithic-ap-node$sd []
```

*** Application Environment Configuration ***

Environment configuration means externalizing application settings (like DB, API keys, ports) instead of hardcoding them in code.

In simple terms:

"Don't put config inside code — pass it from outside."

Issues:
- Not secure
- Hard to change across environments
- Not scalable

Now: Default PORT : 5000

Container runs on 5001

Host also 5001

And PORTs are dynamic

sudo docker run -d -p 5004:5004 -e PORT=5004 myapp

--> -e PORT=5004: -> Sets environment variable inside container - as PORT=5004

*** Debugging Container Builds ***

1. File missing - Fix: Check the Correct file ->

2. Wrong path - Fix: check the file path

3. Dependency Errors - ModuleNotFoundError: No module named 'flask'

Fix:
RUN pip install -r requirements.txt

4. Port / App Not Accessible

Causes:
- Wrong port mapping
- App using localhost

FIX: Check the container port, Host Port & application port then run the Docker

docker run -d -p 5000:5000 myapp

And also in the Application Code:

app.run(host="0.0.0.0", port=5000)

5. Inspect the Container:

docker inspect <container_id>

docker inspect 8390d19ff215

6. Detailed - history:

docker history myapp

** Real Industry Case Study **
Scenario: Production Failure *
- App crashes
- Not responding

Investigation:
1. docker ps → container running

2. docker logs → DB connection error

3. Root cause → wrong env variable

Common Mistakes

Mistake	Fix
Using localhost	Use 0.0.0.0
Wrong port	Match -p
Missing files	Check COPY
Cache issues	Use --no-cache
Dependency issues	Install properly

*** optimize Dockerfile - Multi Stage Docker Build ***

- Reduce final image size by separating build and runtime stages - Multi Stage Docker Build

** Node.js app

*** Tomorrow's tasks ***

Task1: take sample Monolithic application - like banking application then Containerize it

Task 2: Use Multi-Stage Build with Dockerfile, deploy app in production & display the result